



OpenSearch

검색엔진 원리부터 하이브리드 검색까지

역색인 · BM25 · 벡터 검색 · 하이브리드(RSP) · Elasticsearch vs OpenSearch

RDB로 검색하면 왜 느릴까?

```
SELECT * FROM papers  
WHERE title LIKE '%검색어%'
```

모든 행을 하나씩 다 훑는다
(full table scan)

데이터가 쌓일수록
검색이 선형으로 느려짐



그래서 빅데이터 생태계에선
검색엔진을 쓴다

검색엔진은 데이터를 미리 **역색인(inverted index)**해서, 훑지 않고 바로 찾는다.

역색인 — 관계를 뒤집는다

일반적 사고

문서 → 그 안의 단어들

역색인

단어 → 그 단어가 든 문서 목록



책 뒤 '찾아보기'랑 똑같다

단어를 치면 몇 페이지에 나오는지 바로 뜬다.
책 전체를 처음부터 읽을 필요가 없다.

LIKE = 전부 훑기(느림)

역색인 = 단어로 바로 문서 찾기

이 역색인을 실제로 구현한 게 **Lucene**(자바 라이브러리) — OpenSearch-Elasticsearch는 그걸 감싼 서버.

▶ 그럼 뭐가 제일 관련 있는지는 어떻게 알까?

TF-IDF — 단어의 중요도 재기

BM25를 이해하려면 그 출발점인 TF-IDF부터. 두 값을 곱해 "이 단어가 이 문서에서 얼마나 중요한가"를 잰다.

TF

Term Frequency

이 단어가 이 문서에 몇 번 나왔나

많이 나올수록 중요하다고 본다

IDF

Inverse Document Frequency

전체 문서 중 몇 곳에 나오나

드물게 나오는 단어일수록 가중치 ↑

"이다·the"는 흔해서 변별력 X / "트랜스포머"는 희귀 → 강한 신호

TF × IDF = 자주 나오면서(TF) + 다른 문서엔 드문(IDF) 단어가 그 문서의 핵심어

▶ 두 가지 약점: TF 포화, 길이 보정 X

BM25 — 관련도로 줄 세우기

역색인 = 찾기

단어가 든 문서 후보를 빠르게 긁어옴 (순서 없음)

BM25 = 줄 세우기

후보들을 관련도 순으로 점수 매겨 정렬
점수 재료(단어 빈도 등)는 이미 역색인 안에 있음

▶ 근데 단어가 안 맞으면 못 잡는다...

TF-IDF의 개선판

- 희귀한 단어일수록 가중치 ↑
- 단어가 많이 나와도 점수가 포화 (무한정 X)
- 문서가 길면 길이 보정

$k1 \cdot b$ = 튜닝 손잡이

핵심: 단어가 정확히 일치할 때 강하다.

임베딩 벡터 검색 — 의미로 찾기

BM25의 한계

“동근이”

로 검색하면

“헬스” 관련 내용이 안 나온다

단어가 안 겹치니까.



텍스트를 '의미 좌표'로

텍스트 → 벡터(임베딩). 뜻이 가까우면 공간상 위치도 가깝다.

'왕 - 남자 + 여자 = 여왕'

— 의미가 좌표가 되는 것.

k-NN: 쿼리 벡터와 가장 가까운 문서 벡터 찾기

의미로 검색 → 단어가 안 맞아도 됨. 대신 정확한 용어·코드엔 약함. ▶ 그럼 둘 다 쓰면?

둘의 약점을 서로 메운다

BM25

어휘 검색
정확한 제목·용어·저자

+

벡터

시맨틱 검색
의미·동의어·문맥 질의

=

HYBRID

점수 합치는 문제: BM25 점수와 벡터 점수는 스케일이 달라서, 정규화하거나 순위로 합친다(RRF 같은 방식). — *깊이는 생략, 개념만.*

RSP — 논문 탐색 플랫폼

배운 이론이 실제 프로젝트에서 이렇게 돌아갔다 — 3개 신호를 결합한 하이브리드

1

BM25

정확한 논문 제목·저자·전문용어 매칭

2

벡터

“신경망이 문맥 이해하는 법” 같은 의미 질의

3

인용수 점수

신뢰도 신호를 랭킹에 가중

쿼리 → BM25 + 벡터 각각 검색 → 점수 결합(정규화/RRF) → 인용수 가중 → 최종 랭킹

엔진은 같고, 껍데기만 갈라진 형제

둘 다 **Lucene**을 엔진으로 써서 매우 비슷하다.

2021

Elastic이 라이선스 변경

오픈소스(Apache 2.0) → 아니게 됨

2021

AWS가 포크 → OpenSearch 탄생

마지막 오픈소스 버전을 떠서 독립

2024.8

Elastic, AGPL 추가

'오픈소스' 타이틀 회복 — 단, 옵션을 더한 것

이후

둘은 각자 간다

이미 갈라진 뒤라 포크는 안 돌아옴

그래서 뭘 골라야 하나

OpenSearch

완전 오픈소스 (Apache 2.0)

보안·알림·이상탐지까지 전부 무료

AWS 궁합 (Amazon OpenSearch Service)

Elasticsearch

최신 기능이 먼저 들어오는 편

상용 지원·성숙한 생태계(Kibana)

멀티클라우드 매니지드(Elastic Cloud)



RSP에서 OpenSearch를 고른 이유

무료 기능 범위가 넓어 예산에 유리

검색 말고 다른 용도

사실 실무에선 “검색”보다 이쪽으로 더 많이 쓰인다. 오늘 다룬 검색은 그중 하나.



로그 분석 · 관찰성 ★

서버·앱 로그를 모아 검색·시각화하는 ELK 스택. 실무에서 가장 큰 용도.



집계 · 분석 엔진

대량 데이터의 실시간 통계·대시보드 (시간대별·국가별 집계).



알림 (Alerting)

조건을 걸어두고 자동 알림. 예: 에러율 5% 초과 시.



RAG 벡터 백엔드 ★

LLM 임베딩 저장소. RAG 붐을 타고 요즘 급 부상하는 용도.



이상 탐지

ML로 평소와 다른 패턴을 자동 감지. 트래픽·이상 거래 등.



보안 분석 (SIEM)

보안 로그를 모아 위협을 탐지. Security Analytics.

핵심 한 줄: OpenSearch는 “검색엔진”이자 로그 분석·RAG 백엔드까지 겸하는 데이터 플랫폼. 오늘은 그중 검색에 집중했다.

Q&A